

> Question 1: Exception Handling - Robust Calculator

↳ 1 cell hidden

> Question 2: File Operations - Text File Manager

↳ 1 cell hidden

✓ Question 3: CSV File Processing with Error Handling

```
import csv

with open("employees.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(
        ["name", "age", "email", "salary"]
    )

    writer.writerow(
        ["John Doe", 30, "john@example.com", 50000]
    )

    writer.writerow(
        ["Jane Smith", 25, "jane@example.com", 45000]
    )

    writer.writerow(
        ["Bob Johnson", "abc", "bob@example.com", 60000]
    )

    writer.writerow(
        ["Alice Brown", 28, "", 55000]
    )

    writer.writerow(
        ["Charlie Wilson", 35,
         "charlie@example.com", 70000]
    )

print("employees.csv created successfully.")
```

employees.csv created successfully.

```
import csv
import re

print("=== CSV File Processor ===")

input_file = "employees.csv"
output_file = "employees_valid.csv"

valid_records = []
invalid_records = 0
total_salary = 0

try:

    print(f"\nReading {input_file}...")

    with open(input_file, "r", newline="") as file:

        reader = csv.DictReader(file)

        records = list(reader)

        print(
```

```

    f"Processing {len(records)} records..."
)

for row_number, row in enumerate(records, start=1):

    try:
        name = row["name"].strip()
        email = row["email"].strip()

        if not name:
            print(
                f"Row {row_number}: "
                f"Missing name - Skipping row"
            )
            invalid_records += 1
            continue

        if not email:
            print(
                f"Row {row_number}: "
                f"Missing email - Skipping row"
            )
            invalid_records += 1
            continue

        email_pattern = (
            r"^[A-Za-z0-9._%+-]+"
            r"@[A-Za-z0-9.-]+"
            r"\.[A-Za-z]{2,}$"
        )

        if not re.match(
            email_pattern,
            email
        ):
            print(
                f"Row {row_number}: "
                f"Invalid email format - Skipping row"
            )
            invalid_records += 1
            continue

        try:
            age = int(row["age"])
        except ValueError:

            print(
                f"Row {row_number}: "
                f"Invalid age "
                f"'{row['age']}' "
                f"- Skipping row"
            )

            invalid_records += 1
            continue

        try:
            salary = float(row["salary"])
        except ValueError:

            print(
                f"Row {row_number}: "
                f"Invalid salary "
                f"'{row['salary']}' "
                f"- Skipping row"
            )

            invalid_records += 1
            continue

        valid_records.append(
            {
                "name": name,
                "age": age,
                "email": email,
                "salary": salary
            }
        )

```

```

        total_salary += salary

    except KeyError as e:

        print(
            f"Row {row_number}: "
            f"Missing column {e}"
        )

        invalid_records += 1

valid_count = len(valid_records)

average_salary = (
    total_salary / valid_count
    if valid_count > 0
    else 0
)

print("\n=== Processing Complete ===")

print(
    f"Valid Records: {valid_count}"
)

print(
    f"Invalid Records: {invalid_records}"
)

print("\nStatistics:")

print(
    f"Average Salary: "
    f"Rs {average_salary:,.2f}"
)

print(
    f"Total Employees: "
    f"{valid_count}"
)

print(
    f"\nWriting valid data "
    f"to '{output_file}'..."
)

with open(
    output_file,
    "w",
    newline=""
) as file:

    fieldnames = [
        "name",
        "age",
        "email",
        "salary"
    ]

    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames
    )

    writer.writeheader()

    writer.writerows(valid_records)

print(
    "File written successfully."
)

except FileNotFoundError:

    print(
        f"Error: File "

```

```

        f"'{input_file}' not found."
    )

except PermissionError:

    print(
        "Error: Permission denied."
    )

except csv.Error as e:

    print(
        f"CSV Error: {e}"
    )

except IOError as e:

    print(
        f"I/O Error: {e}"
    )

except Exception as e:

    print(
        f"Unexpected Error: {e}"
    )

```

```

=== CSV File Processor ===

```

```

Reading employees.csv...
Processing 5 records...
Row 3: Invalid age 'abc' - Skipping row
Row 4: Missing email - Skipping row

```

```

=== Processing Complete ===
Valid Records: 3
Invalid Records: 2

```

```

Statistics:
Average Salary: Rs 55,000.00
Total Employees: 3

```

```

Writing valid data to 'employees_valid.csv'...
File written successfully.

```

Question 4: JSON Configuration Manager

```

import json
import os

CONFIG_FILE = "config.json"

DEFAULT_CONFIG = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "mydb"
    },
    "api": {
        "timeout": 30,
        "retries": 3
    },
    "logging": {
        "level": "INFO",
        "file": "app.log"
    }
}

def validate_config(config):

    required_sections = [
        "database",
        "api",
        "logging"
    ]

```

```

for section in required_sections:

    if section not in config:
        raise ValueError(
            f"Missing required section: {section}"
        )

if not isinstance(
    config["database"]["host"],
    str
):
    raise TypeError(
        "Database host must be string"
    )

if not isinstance(
    config["database"]["port"],
    int
):
    raise TypeError(
        "Database port must be integer"
    )

if not (
    1 <= config["database"]["port"] <= 65535
):
    raise ValueError(
        "Port must be between 1 and 65535"
    )

if not isinstance(
    config["api"]["timeout"],
    int
):
    raise TypeError(
        "Timeout must be integer"
    )

if config["api"]["timeout"] <= 0:
    raise ValueError(
        "Timeout must be positive"
    )

return True

def load_config():

    try:

        print(
            f"Loading {CONFIG_FILE}..."
        )

        with open(
            CONFIG_FILE,
            "r"
        ) as file:

            config = json.load(file)

            validate_config(config)

            print(
                "Configuration loaded successfully."
            )

            return config

    except FileNotFoundError:

        print(
            "Config file not found."
        )

        print(

```

```

        "Creating default configuration..."
    )

    save_config(DEFAULT_CONFIG)

    return DEFAULT_CONFIG

except json.JSONDecodeError:

    print(
        "Error: Invalid JSON format "
        "in file."
    )

    print(
        "Using default configuration."
    )

    return DEFAULT_CONFIG

except Exception as e:

    print(
        f"Configuration Error: {e}"
    )

    print(
        "Using default configuration."
    )

    return DEFAULT_CONFIG
def save_config(config):

    try:

        with open(
            CONFIG_FILE,
            "w"
        ) as file:

            json.dump(
                config,
                file,
                indent=4
            )

            print(
                f"Configuration saved to "
                f"{CONFIG_FILE}"
            )

    except IOError as e:

        print(
            f"Write Error: {e}"
        )

def display_config(config):

    print("\nCurrent Settings:")

    print(
        f"- Database Host: "
        f"{config['database']['host']}"
    )

    print(
        f"- API Timeout: "
        f"{config['api']['timeout']} seconds"
    )

    print(
        f"- Logging Level: "
        f"{config['logging']['level']}"
    )

```

```

print(
    "=== JSON Configuration Manager ==="
)

config = load_config()

display_config(config)
print(
    "\nUpdating API timeout to 60..."
)

config["api"]["timeout"] = 60

print(
    "Configuration updated."
)

print(
    "\nAdding new setting:"
    " cache.enabled = True"
)

config["cache"] = {
    "enabled": True
}

print(
    "New setting added."
)
print(
    "\nRemoving logging file setting..."
)

config["logging"].pop(
    "file",
    None
)

print(
    "Setting removed."
)
print(
    "\nSaving configuration..."
)

save_config(config)

# -----
# Error Handling Test
# -----

print(
    "\n=== Error Handling Test ==="
)

try:

    with open(
        "invalid.json",
        "w"
    ) as file:

        file.write(
            "{ invalid json"
        )

    with open(
        "invalid.json",
        "r"
    ) as file:

        json.load(file)

except json.JSONDecodeError:

    print(

```

```
"Error: Invalid JSON format "  
"in file."  
)  
  
print(  
    "Using default configuration "  
  
=== JSON Configuration Manager ===  
Loading config.json...  
Config file not found.  
Creating default configuration...  
Configuration saved to config.json  
  
Current Settings:  
- Database Host: localhost  
- API Timeout: 30 seconds  
- Logging Level: INFO  
  
Updating API timeout to 60...  
Configuration updated.  
  
Adding new setting: cache.enabled = True  
New setting added.  
  
Removing logging file setting...  
Setting removed.  
  
Saving configuration...  
Configuration saved to config.json  
  
=== Error Handling Test ===  
Error: Invalid JSON format in file.  
Using default configuration.
```

› Question 5: Logging System with File Management

↳ 1 cell hidden

› Question 6: Data Backup System

↳ 1 cell hidden

› Question 7: File Validator and Processor

↳ 1 cell hidden

› Question 8: Comprehensive File Management System

↳ 1 cell hidden